GOLD: More Selection and Switch (Fri Sep 4)

Swapping Variables

Problem: Swapper

Write a program that, given two values in variables x and y, ensures that y is not less than x (swap them if necessary)

What's wrong with:

```
if (y < x) {
    x = y;
    y = x;
}
```

Code: (constructed during class)

```
void main () {
    String s1 = IO.readln("Enter x: ");
    String s2 = IO.readln("Enter y: ");
    int x = Integer.parseInt(s1);
    int y = Integer.parseInt(s2);
    IO.println(String.format("BEFORE: x = %d, y = %d",x,y));
    if (y < x) {
        int tmp = x;
        x = y;
        y = tmp;
    }
    IO.println(String.format("AFTER: x = %d, y = %d",x,y));
}
```

Median

Median

Problem: Write a method that takes 3 doubles as input and returns the median (middle) number.

We will illustrate cascading if-else-if with compound boolean expressions.

Code: (constructed during class)

```
double median(double x, double y, double z) {
    if ((y <= x && x <= z) || (z <= x && x <= y)) {
        return x;
    } else if ((x <= y && y <= z) || (z <= y && y <= x)) {
        return y;
    } else {
        return z;
    }
}

void main() {
    IO.println(median(3.0, 4.0, -2.0));   // 3.0
    IO.println(median(12, 14, 27));       // 14.0
    IO.println(median(24, 15, 18));       // 18.0
}
```

Why is a plain `else` enough for the last branch? "If x isn't the middle one and y isn't either, then it must be z."

This argument has a hidden assumption: **every pair of doubles compares as `x <= y` or `y <= x`.** Is this true?

No. In Java, `doubles` don't quite match our conception of real numbers:

- Dividing by 0.0 is not an exception like with integers, instead we get `NaN` ("not a number", e.g. `0.0 / 0.0`)
- When `NaN` is involved, every comparison evaluates to **false.**
- All three cases can miss, and the plain `else` quietly returns `z` as if nothing happened.

The correct version:

```
double median(double x, double y, double z) {
    if ((y <= x && x <= z) || (z <= x && x <= y)) {
        return x;
    } else if ((x <= y && y <= z) || (z <= y && y <= x)) {
        return y;
    } else if ((x <= z && z <= y) || (y <= z && z <= x)) {
        return z;
    }
    // "unreachable" -- IF every pair of doubles compares. NaN doesn't.
    return 0.0 / 0.0;   // explicit NaN: there is no median
}

void main() {
    IO.println(median(3.0, 4.0, -2.0));   // 3.0
    IO.println(median(12, 14, 27));       // 14.0
    IO.println(median(24, 15, 18));       // 18.0
}
```

▶ Expand

Review: Substring

String methods: substring

Suppose we want to extract the extension of a file name.

```
String name = "fluffy.jpg";
String ext  = name.substring(6,10);
```

- substring arguments are `[begin, end)`
- `[0,10)` starts at 0 and goes up to but does not include 10
- Example: `[6,10)` is 6, 7, 8, 9

`name` contains`fluffy.jpg`
`ext` contains `.jpg`

File Extensions

Problem: Write a method

`String findExtension(String filename)`

that finds the extension in String filename.

Methods: What methods should we use?

- `.substring(beg, end)` will give us the extension. But where does it start?
- We can use `.indexOf(ch)` to find the first `.` (the one that starts the extension)

Code:

Let's draft the code together:

```
String findExtension(String filename) {

}

void main() {

}
```

What can go wrong above?

Empty Strings and Null Pointers

If we define:

`String fname = "fluffy.jpg";`

`fname` points to the object that contains the string `"fluffy.jpg"`



If

```
String fname = "";
```

`fname` points to the object that contains the string `""` (empty string)



Finally, if

```
String fname;
```

or

```
String fname = null;
```

`fname` does not point to a string. It is a null pointer.



No methods can be called on a null pointer. This is an exception (error) that crashes the program.

We always want to avoid this!

Let's fix the Code

Code version 2: Let's make the code safe:

```
String findExtension(String filename) {
    if (filename == null)   // why not filename.equals(null)?
        return "";          //      can't call methods on null values

    int dot = filename.indexOf('.');

    if (dot == -1)
        return "";

    return filename.substring(dot); // or (dot, filename.length())
}

void main() {
    String example = "lecture_notes.pdf";
    IO.println(findExtension(example));

    IO.println("Extension is: " +
        findExtension(IO.readln("Enter your filename: ")));

    IO.println(findExtension(null));
}
```

For reference — a more compact, idiomatic version:

▶ Expand

Pause and Predict: Substring

Pause and predict: substring

Three quick predictions on `"racecar"` (indexes 0–6). Commit to an answer for each.

Q1

What is `"racecar".substring(0, 4)`?

- **A.** `"rac"`
- **B.** `"race"`
- **C.** `"racec"`
- **D.** `"acec"`

Q2

What is `"racecar".substring(4, 7)`?

- **A.** `"eca"`
- **B.** `"ecar"`
- **C.** `"car"`
- **D.** `"ceca"`

Q3

Which call returns `"aceca"`?

- **A.** `"racecar".substring(1, 5)`
- **B.** `"racecar".substring(1, 6)`
- **C.** `"racecar".substring(1, 7)`
- **D.** `"racecar".substring(2, 5)`
- **E.** `"racecar".substring(2, 6)`

Reading: The Dangling Else

The Dangling Else Problem

We talked about how if/else blocks can be nested inside each other.

This can lead to ambiguity and unexpected behavior if we don't use braces:

Suppose we write the following code, and we intend it to work as indented:

```
if (familyVisiting)
    if (isWarmOut)
        goToPark();
else
    hangoutWithFriends();
```

- **But it does not work like this!** The else clause goes with the most recent if, not as formatted above.

- What really happens is the following:

```
if (familyVisiting)
    if (isWarmOut)
        goToPark();
    else
        hangoutWithFriends();
```

- **Best practice:** Use braces to disambiguate

Ternary Assignments

A common situation is to assign one of two alternative values depending on a condition:

```
if (a < b)
    minVal = a;
else
    minVal = b;
```

We can use the following to accomplish this in one line:

```
minVal = (a < b) ? a : b;
```

Recall the solution to the Tutor Pay problem:

```
double computePay(double hours) {
    double pay;
    if (hours > 20)
        pay = 18.00 * 20 + (hours - 20) * 25.00;
    else
        pay = hours * 16.00;
    return pay;
}
```

We can write this more compactly with a ternary assignment:

```
double computePay(double hours) {

}
```

Leap Year

Implement the boolean method `isLeapYear` to determine whether a non-negative integer year is a leap year.

A year is a leap year if it is divisible by 4 (`year % 4 == 0`) **unless** it is divisible by 100. However, one last exception: years divisible by 400 **are** leap years after all.

So:

- 2024 → leap year (divisible by 4)
- 2026 → not a leap year
- 2100 → **not** a leap year (divisible by 100)
- 2000 → leap year (divisible by 400)

The starter code's `return false;` is a placeholder so the program runs. Replace it with your logic, then press **Test**.

Keep your finished `isLeapYear` handy: the switch example later in the lecture calls it.

Switch Syntax

Switch Statements

When we have a bunch of alternatives to consider, then `if` ... `else if` ... `else if` ... `else` blocks can be cumbersome.

The switch statement takes an expression in parentheses which can be an `int` (or other whole number type), `char`, or `String` .

We have one or more `case` branches which have a list of one or more literals or constants followed by an `->` and a block.

An optional `default` block is used when no case matches:

```
switch (expression) {
    case value, …, value -> { statements; }
    case value, …, value -> { statements; }
    …
    default -> { statements; }
}
```

Using Switch

Using Switch

Using if-else with many branches isn't always pretty. Suppose we want the number of days in a month (using the `isLeapYear` method you just in the exercise above):

```
int daysInMonth1(int month, int year) {
    if (month == 1) {                // January
        return 31;
    } else if (month == 2) {
        if (isLeapYear(year)) {
            return 29;
        } else {
            return 28;
        }
    } else if (month == 3) {
        return 31;
    } else if (month == 4) {
        return 30;
    } else if (month == 5) {
        return 31;
    } else if (month == 6) {
        return 30;
    } else if (month == 7) {
        return 31;
    } else if (month == 8) {
        return 31;
    } else if (month == 9) {
        return 30;
    } else if (month == 10) {
        return 31;
    } else if (month == 11) {
        return 30;
    } else if (month == 12) {
        return 31;
    }
    return -1;
}
```

The modern alternative is the **arrow switch**. Let's rewrite the ladder together:

```
boolean isLeapYear(int year) {
    return (year % 4 == 0) && (year % 100 != 0 || year % 400 == 0);
}

int daysInMonth(int month, int year) {

    return -1;
}

void main() {
    IO.println(daysInMonth(2, 2024));   // 29
    IO.println(daysInMonth(9, 2026));   // 30
}
```

The right-hand side of each arrow must be a statement or a block. Cases can list **several values**, separated by commas.

Switch expressions

Often every branch computes a different value for the **same variable**. Then `switch` can be used as an **expression.** In other words, it evaluates to a value:

```
boolean isLeapYear(int year) {
    return (year % 4 == 0) && (year % 100 != 0 || year % 400 == 0);
}

int daysInMonth(int month, int year) {
    int days = switch (month) {
        case 1, 3, 5, 7, 8, 10, 12 -> 31;
        case 4, 6, 9, 11 -> 30;
        case 2 -> {
            if (isLeapYear(year)) {
                yield 29;
            } else {
                yield 28;
            }
        }
        default -> -1;
    };                                  // the expression form ends in a semicolon
    return days;
}

void main() {
    IO.println(daysInMonth(2, 2024));   // 29
    IO.println(daysInMonth(2, 2100));   // 28
}
```

An expression evaluates to a value, so instead of blocks of statements on the right hand side of the arrows, we need a value or something that produces a value:

- a normal Java expression like `31` or `s.length() + x + 3`
- a block that computes a value returned with `yield x;` where x is a value. (not `return`)
- and the switch must cover **every** possible value. That's why `default` is **required** here.

We can mix expressions and blocks on the right-hand side, but a block must use the keyword `yield` to "return" a value to the expression.

Pause and predict (poll)

Q. What happens with this code?

```
int code = 2;
String s = switch (code) {
    case 1 -> "one";
    case 2 -> { yield "two"; }
    case 3 -> "three";
};
IO.println(s);
```

- **A.** It prints `two`
- **B.** It prints `one`
- **C.** Compile error
- **D.** It compiles, but crashes at run time

Switch on Strings

Switch on Strings

`switch` also works on `String`s. It compares **contents**, like `.equals`, so it's safe for text.

Example: Recruiter Swag. First-years get gloves and a coat, sophomores a coat, juniors and seniors a backpack:

```
String handoutFor(String yearName) {
    return switch (yearName.toUpperCase()) {
        case "FRESHMAN" -> "gloves and a coat";
        case "SOPHOMORE" -> "a coat";
        case "JUNIOR", "SENIOR" -> "a backpack";
        default -> "nothing";
    };
}

void main() {
    IO.println(handoutFor("sophomore"));  // a coat
    IO.println(handoutFor("Junior"));     // a backpack
    IO.println(handoutFor("grad"));       // nothing
}
```

- A case matches the String **exactly.**
  - `"junior"` would fall to `default`.
  - So **normalize first**: `toUpperCase()` from Monday makes one case cover every capitalization.
- The expression form + `default` guarantees `handoutFor` always returns something.

The Old Switch

The old switch (recognize, don't write)

Before arrows, Java's switch used colons and `break`. You will meet it in older code, on the web, and in AI output:

```
void main() {
    int n = 1;
    switch (n) {
        case 1: IO.println("one"); break;
        case 2: IO.println("two"); break;
        default: IO.println("many"); break;
    }
}
```

⚠️

Without `break`, execution **falls through** into the next case; matching `1` would then print all three lines. Forgotten `break`s are a classic bug; the arrow form simply cannot have it.

In this course: read the colon form, write arrows.

Take Home: Letter Grades

Today you met two tools for choosing between alternatives: the **else-if ladder** and the **switch**. This exercise makes you use each one.

Implement **both** methods:

1. `String letterFor(int score)` : convert a score (0–100) to a letter on the simple scale:

- 90 and above → `"A"`
- 80–89 → `"B"`
- 70–79 → `"C"`
- 60–69 → `"D"`
- below 60 → `"F"`

Score *ranges* want a **ladder;** try writing this as a switch and you'll see why it doesn't fit here.

2. `double pointsFor(String letter)` : convert a letter grade to grade points:

- `"A"` → 4.0, `"B"` → 3.0, `"C"` → 2.0, `"D"` → 1.0, `"F"` → 0.0
- **any capitalization counts**: `"b"` is worth 3.0
- anything that isn't a grade → `-1.0` (a signal value, like `indexOf`'s -1)

Discrete *labels* want a **switch:** normalize first and remember: a switch expression must cover every possible input.

Press **Test** to run the tests.

As always, remember the boundaries: is 90 an A? Is 59 a D? Is `"f"` a real grade or garbage?